

Web Security - Introduction

**CSCI 4130: Introduction to Computer
Security**

Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

The World Wide Web

让全球范围内的用户能够通过互联网访问和检索分散在不同地理位置的信息

- The World Wide Web was originally conceived as a geographically distributed information retrieval system

允许用户通过点击链接从一个文档跳转到另一个相关文档

- The documents are interconnected through hyper links

支持动态交互和复杂应用（如在线办公、社交媒体、电子商务等）

- The Web has evolved into a full-fledged application platform for the execution of distributed applications

- The Web is also vulnerable to many attacks

- The impact of these attacks is enormous

渗透到生活的方方面面（如银行、医疗、政府服务），攻击可能导致数据泄露或服务中断。

- Widespread use of the service

服务器通常24/7在线，容易成为持续攻击的目标

- Accessibility of the servers

大量用户设备（如手机、电脑）可能成为攻击入口或受害者。

- Widespread use of the clients

Web Security Goals

- Confidentiality

确保恶意网站无法窃取用户计算机或其他网站上的数据

- Malicious web sites shall not be able to learn the data on my computer and on other web sites

- Integrity

防止恶意网站篡改用户计算机或网站的数据

- Malicious web sites shall not be able to modify the data on my computer and on other web sites

- Authenticity

防止恶意网站冒充合法网站

- Malicious web sites shall not be able to impersonate another website

Web Security Goals (Cont.)

- Availability

攻击者无法通过攻击使目标网站不可用（如DDoS攻击）

- Attackers cannot make a website we visit unavailable

- Privacy

防止恶意网站追踪用户的在线行为（如浏览记录、地理位置）

- Malicious websites should not be able to learn my activities online

- Computing resource

防止恶意网站耗尽或滥用用户设备的计算资源（如挖矿脚本、无限循环攻击）

- Malicious websites should not be able to exhaust or abuse my computer's computing power
 - Another form of availability

- Others?

Web Vulnerability Analysis

Web漏洞分析

- Vulnerabilities
 - In the protocols
 - In the infrastructure
 - In the server-side (back-end) of the application
 - In the client-side (front-end) of the application
- Many vulnerabilities are the results of interactions of the various components used for processing a request
- Understanding the basic technology is key

Web请求的处理涉及多组件协作（如浏览器→负载均衡→服务器→数据库），任一环节的缺陷或交互不当都可能引发漏洞。

Web漏洞有 4 大来源

(1) 协议层漏洞 (In the protocols)

- **定义：**与Web通信协议（如HTTP/HTTPS、DNS、TCP/IP）相关的漏洞。
- **典型例子：**
 - **HTTP明文传输：**数据可被中间人窃听（需HTTPS修复）。
 - **TLS协议漏洞：**如心脏出血（Heartbleed）漏洞可泄露服务器内存数据。
- **影响：**协议层漏洞通常影响范围广，修复需升级底层基础设施。

(2) 基础设施漏洞 (In the infrastructure)

- **定义：**支撑Web运行的硬件、网络设备或中间件（如Web服务器、数据库、CDN）的漏洞。
- **典型例子：**
 - **Nginx/Apache配置错误：**导致目录遍历或信息泄露。
 - **过时的DNS服务器：**易受缓存投毒攻击。
- **影响：**基础设施漏洞可能被利用以瘫痪服务或获取控制权。

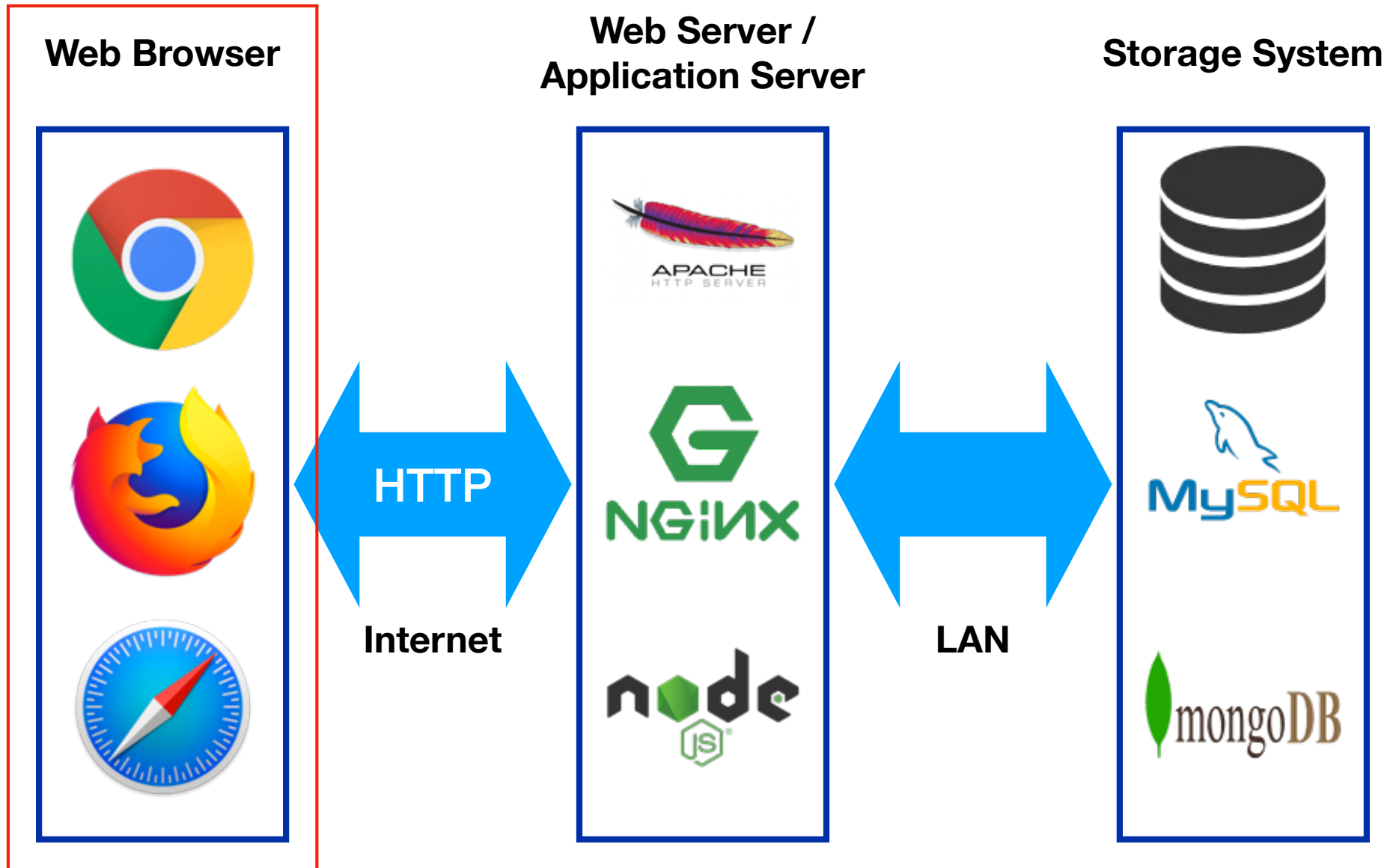
(3) 服务端漏洞 (Server-side/Back-end)

- **定义：**后端应用程序代码或逻辑的漏洞（如业务逻辑、数据库交互）。
- **典型例子：**
 - **SQL注入：**通过输入恶意SQL语句篡改数据库。
 - **文件上传漏洞：**上传恶意文件获取服务器权限。
- **影响：**可直接导致数据泄露、服务器沦陷。

(4) 客户端漏洞 (Client-side/Front-end)

- **定义：**前端代码（如HTML/JavaScript）或用户浏览器环境的漏洞。
- **典型例子：**
 - **跨站脚本 (XSS)：**注入恶意脚本窃取用户Cookie。
 - **CSRF (跨站请求伪造)：**诱导用户执行非预期操作（如转账）。
- **影响：**主要危害用户数据与隐私。

Web Application Architecture



URIs, URLs, URNs

- A Uniform Resource Identifier is a string that identifies a resource
 - Scheme: Protocol/framework
 - Authority: A name space that qualifies the resource
 - Path: A pathname composed of “/” separated strings
 - Query: An application-specific piece of information
- A URI can be further classified as a locator, a name, or both
 - 定位器
 - 名称

- **Scheme (方案)** : 指定协议或框架 (如 `http`、`ftp`、`mailto`) 。
 - 示例: `https://` 表示使用HTTPS协议。
- **Authority (授权机构)** : 定义资源的命名空间 (通常包括域名和端口) 。
 - 示例: `www.example.com:80` 。
- **Path (路径)** : 以 `/` 分隔的字符串, 表示资源在服务器上的位置。
 - 示例: `/products/index.html` 。
- **Query (查询参数)** : 应用程序特定的信息, 以 `?` 开头, 键值对形式 (如 `?id=123`) 。

- **核心特点:**
 - 必须包含如何获取资源的详细信息 (如协议、域名、路径) 。
 - 资源位置变化时, URL失效。
- **示例:**
 - `https://www.example.com/images/logo.png`
 - Scheme: `https`
 - Authority: `www.example.com`
 - Path: `/images/logo.png`

URIs, URLs, URNs

- A **Uniform Resource Locator** is an identifier that contains the information to locate the resource
 - Where an identified resource is available and the mechanism for retrieving it
- A **Uniform Resource Name** is an identifier that identifies an entity regardless of the fact that it is accessible or even if it exists
 - Example: ISBN

- 定义：URN是URI的子集，仅标识资源名称，与资源的位置和可访问性无关。
- 核心特点：
 - 永久唯一性，即使资源已不存在或无法访问，URN仍有效。
 - 通常以 `urn:` 开头。
- 示例：
 - `urn:isbn:0451450523` (标识一本特定的书籍，无论其电子版或纸质版是否存在)。
 - `urn:uuid:6e8bc430-9c3a-11d9-9669-0800200c9a66` (唯一标识符)。

Hypertext Markup Language (HTML)

- Describes the **structure** of Web pages using markup
- Markup Language
 - Include **directives** with **content**
指令 (标签)
 - Directives can dictate presentation and describe content, e.g., `<title>Title text</title>`
- Web pages are built from **HTML elements** *← 由标签与内容组成*
- **HTML elements** are represented by **tags**
 - HTML uses `<tag>...</tag>` to denote tags
 - Self-closing tag: `<bar />`
 - Void tags: `<img>`

(1) 常规标签 (双标签)

- 语法: `<tag>内容</tag>`
- 示例:

```
html
<h1>Main Heading</h1>  <!-- 定义一级标题 -->
<a href="https://example.com">Link</a>  <!-- 定义超链接 -->
```

(2) 自闭合标签 (Self-closing tag)

- 语法: `<tag />` (XHTML风格) 或 `<tag>` (HTML5风格)。
- 用途: 用于无需内容的元素, 如换行、图片。
- 示例:

```
html
<br />  <!-- 换行 (XHTML) -->
<br>    <!-- 换行 (HTML5) -->
```

1. Void Tags (空标签)

定义:

- 没有闭合标签, 且不能包含任何内容或子元素的标签。
- 是HTML标准中明确规定的特殊标签类别。

特点:

- 语法: 只需写开始标签, 无需闭合 (如 `<img>`、`<br>`)。
- 常见用途: 用于插入外部资源 (如图片、链接) 或功能性标记 (如换行、水平线)。
- 示例:

```
html

<br>    <!-- 换行 -->
<input type="text">
```

为什么叫“Void”? :

- “Void”意为“空”, 表示这些标签内部永远为空 (不能像 `<div>` 那样嵌套内容)。

常见Void Tags:

标签	用途
<code></code>	插入图片
<code>
</code>	换行
<code><hr></code>	水平线
<code><input></code>	输入框
<code><meta></code>	元数据 (如字符集声明)
<code><link></code>	引入外部资源 (如CSS)

Attributes of HTML Tags

- Tags can have **attributes** that provide **metadata** about the tag
- They live inside the start tag after the tag name
 - Multiple attributes are separated by spaces

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Page Title</title>
```

```
</head>
```

```
<body>
```

```
<h1>Introduction</h1>
```

```
<p id="p1" style="color: red">Welcome to my page!</p>
```

```
</body>
```

```
</html>
```

内联CSS样式, 设置颜色

为该段落赋予唯一 -

标识符, 便于CSS或JS操作

Introduction

Welcome to my page!

JavaScript

- The programming language of HTML and the Web
 - Supported by all browsers
- It is a high-level programming language
 - Weakly and dynamically typed
 - Interpreted
 - Objects are supported 面向对象
- Very powerful
 - Can manipulate almost all (client-side) web site data

弱类型 (Weakly Typed) :

- 变量类型可隐式转换 (如 `"5" + 2` 得到字符串 `"52"`) 。
- **风险**: 可能导致意外行为, 需谨慎处理类型:

javascript

复制

```
console.log("5" - 2); // 3 (字符串转数字)
console.log("5" + 2); // "52" (数字转字符串)
```

动态类型 (Dynamically Typed) :

- 变量类型在运行时确定, 无需声明类型:

javascript

复制

```
let x = 10; // 数字
x = "hello"; // 变为字符串
```

解释执行:

- 代码逐行被浏览器引擎**实时解释**, 无需预先编译。
- **优势**: 开发快速, 修改后立即生效。
- **劣势**: 性能低于编译型语言 (如Java), 但JIT (即时编译) 技术已大幅优化。

HTML Frames

允许在同一页面显示多个独立的视图或页面，这些视图或页面关联着不同的URL

- The `<frame>` tags allow for displaying multiple separate views or pages that are associated with separate URLs on the same page } 既多页面共存
 - Used in the early days to display navigation controls
 - Frames are enclosed by a `<frameset>` tag
- The `<iframe>` tags are inline frames 内联框架
 - Similar to frames but do not require a `<frameset>` ↓ 允许当前页面嵌入另一个独立文档

特性	<frame>	<iframe>
是否需要frameset	是	否
现代浏览器支持	已废弃	完全支持
加载方式	必须分割整个浏览器窗口	可嵌入页面任意位置
典型用途	早期网站导航框架	嵌入广告/地图/第三方内容

1. 传统 <frameset> + <frame> 示例

特点:

- 已废弃 (HTML5不支持), 但早期用于分割浏览器窗口。
- 需要包裹在 <frameset> 中, 每个 <frame> 加载独立页面。

代码示例:

html 复制

```
<!DOCTYPE html>
<html>
  <frameset cols="25%,75%"> <!-- 将窗口分为左右两列 -->
    <frame src="nav.html" name="navbar"> <!-- 左侧导航栏 -->
    <frame src="content.html" name="main"> <!-- 右侧主内容 -->
  </frameset>
</html>
```

▶ 运行 HTML

效果:

复制

+-----+

| 导航栏 |

| (来自 nav.html) |

+-----+

| 内容区 | 主内容 |

| (静态) | (来自 content.html) |

| | |

+-----+

注意事项:

- 若直接运行, 现代浏览器会显示空白 (已弃用)。
- 早期用途: 左侧固定导航菜单, 右侧动态加载内容。

2. 现代 <iframe> 示例

特点:

- 当前仍广泛使用, 可嵌入任意位置。
- 不需要 <frameset>, 直接嵌入页面中。

代码示例:

html 复制

```
<!DOCTYPE html>
<html>
<body>
  <h1>我的网页</h1>

  <!-- 嵌入一个内联框架 -->
  <iframe
    src="https://example.com"
    width="600"
    height="400"
    title="示例网站"
    sandbox="allow-same-origin"
  ></iframe>

  <p>上方嵌入了 example.com 的内容</p>
</body>
</html>
```

效果:

复制

+-----+

| 我的网页 |

+-----+

| [example.com的完整内容] |

| (以600x400像素窗口显示) |

| |

+-----+

| 上方嵌入了 example.com 的内容 |

+-----+

常见用途:

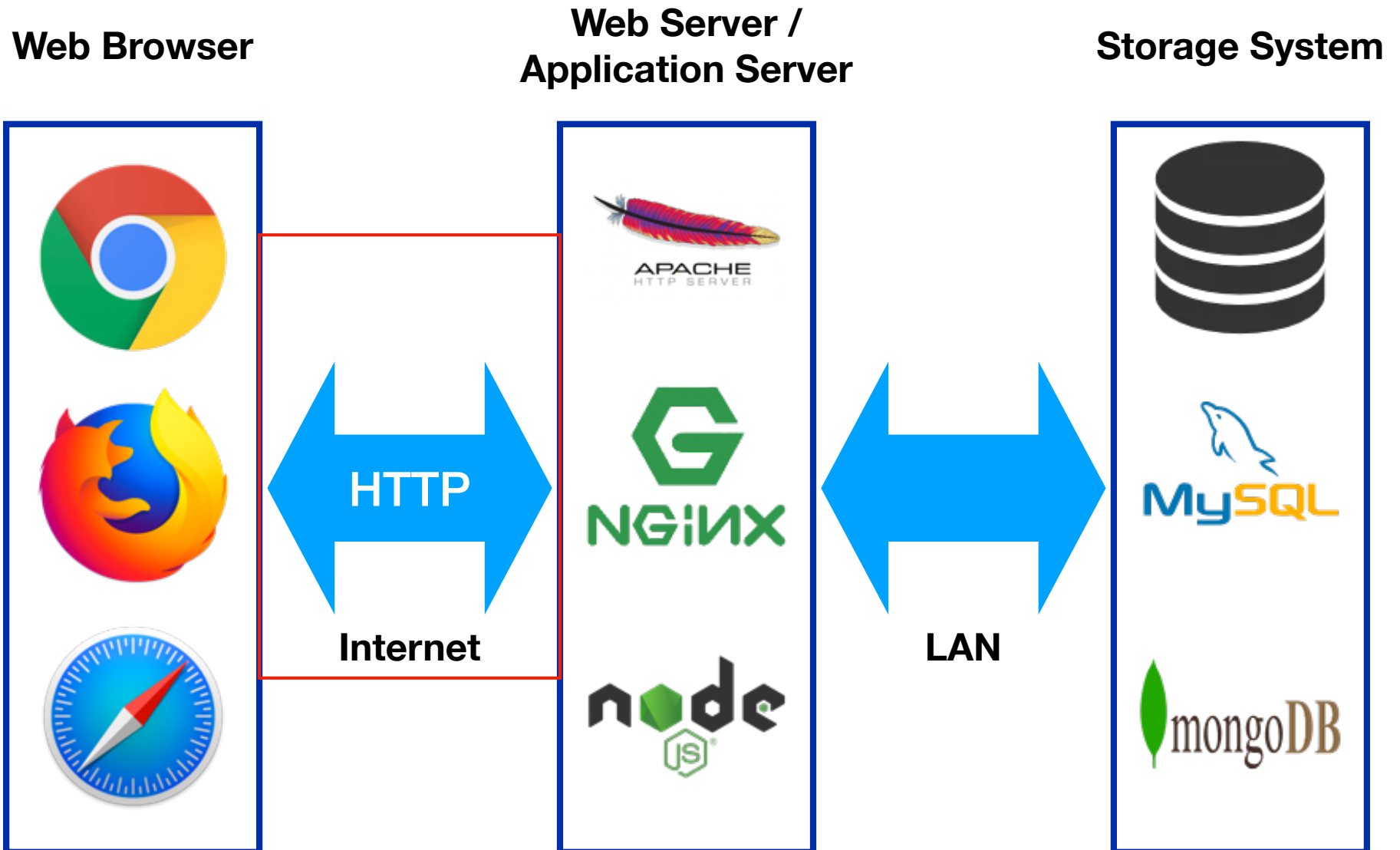
- 嵌入第三方内容 (如YouTube视频、谷歌地图)。
- 沙箱隔离测试 (如在线代码编辑器)。

- 主框架 (Main Frame) :
 - 用户直接访问的页面, 作为其他框架的容器。
- 子框架 (Child Frame) :
 - 由父框架通过 <iframe> 或 <frame> 加载, 受父框架控制:
 - 样式与布局: 父框架通过CSS设置子框架大小、边框等。
 - 内容隔离: 子框架内容独立渲染, 但受同源策略限制。

HTML Frames (Cont.)

- The page we directly visit is loaded in a main frame
- The embedding/parent frame can specify only style and placement of the embedded/child frame in its own frame
- Frames can show content in their own interfaces (rectangles)
 - They cannot draw outside the designated interfaces

Web Application Architecture



既 HTTP, 超文本传输协议

HyperText Transfer Protocol

应用层协议

- An application-layer protocol used to transfer information between a web client and a web server

TCP提供可靠、有序的数据传输（丢包重传、顺序校正），
确保HTTP请求/响应完整送达。

- Uses TCP port 80

客户端

服务器

- Client

Transmission Control Protocol, 是传输层协议, 负责在不安全环境中
提供可靠, 面向连接的数据传输

- Opens a TCP connection and sends requests

- Server

- Accepts the connection and sends replies

- Multiple requests can be sent using the same TCP connection

支持并发处理

HTTP Request

HTTP请求本质是客户端（如浏览器）将格式化文本写入Socket，通过TCP连接发送给服务器。

- Writes data into **socket** → 是网络通信的核心编程接口，充当应用程序与网络协议栈之间的桥梁
- Embeds input data by the user in the headers and the body

- Headers specify

- Method
- Resource
- Protocol version
- Other info
 - General header
 - Request header
 - Entity header

Headers

**Body
(optional)**

Method **URL** **Version**
GET /index.html HTTP/1.1

Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html, */*
Accept-Language: en-us
Accept-Charset: ISO-8859-1,utf-8
Connection: keep-alive

\r\n

Method	请求方法 (如 <code>GET</code> 获取资源, <code>POST</code> 提交数据)。
Resource	请求的资源路径 (如 <code>/index.html</code>)。
Version	HTTP协议版本 (如 <code>HTTP/1.1</code> 支持持久连接)。

头类型	示例	作用
General	<code>Connection: keep-alive</code>	控制TCP连接复用 (避免频繁握手)。
Request	<code>User-Agent: Mozilla/5.0</code>	标识客户端类型 (浏览器、爬虫等)。
Entity	<code>Content-Type: application/json</code>	描述请求体的格式 (如POST提交JSON数据)。

请求头字段	类型	说明
Host	请求头	请求的主机名
User-Agent	请求头	客户端的用户代理信息
Accept	请求头	客户端可接受的 MIME 类型
Accept-Language	请求头	客户端可接受的语言
Accept-Charset	请求头	客户端可接受的字符集编码
Connection	通用头	是否保持连接

HTTP Methods

从服务器获取指定URL标识的资源（如HTML页面、图片、API数据）

- **GET** - **Reads** data from a specified resource referred by the URL

检查资源是否存在（如 200 OK vs 404 Not Found）。

验证文件是否修改（通过 Last-Modified 或 ETag 头）。

预获取资源元数据（如 Content-Length 决定是否下载大文件）。

只请求资源的响应头（Headers），不返回实际数据（Body为空）

- **HEAD** - Fetches **only the information (HTTP header)** about data at a specified resource

向指定URL发送需处理的数据（如表单、文件上传），服务器返回处理结果

- **POST** - **Submits** “data” to be processed with a specified resource identified by the URL and **gets a response** back

- Message posts

- Form data submission

将请求体中的数据完整替换目标URL下的资源（如覆盖整个文件）

- **PUT** - **Uploads** data to be stored under a specified resource

- It is different from the POST method

Resources

- A resource can be specified by an absolute URI or an relative path
- Absolute URIs are used when requesting a resource through a proxy 代理需明确知道目标服务器的完整地址才能转发请求
 - GET http://www.exmaple.com/index.html HTTP/1.1
- Relative paths are used when requesting a resource from a server that owns the resource
 - GET /index.html HTTP/1.1

HTTP Response

- Reads data from socket

Version Status Status Message

HTTP/1.1 200 OK

Date: Sun, 7 Jul 2018 17:36:27 GMT

Server: Apache

Content-Encoding: gzip

Content-Type: text/html; charset=UTF-8

Content-Length: 1846

Headers

这几个是
entity-header

\r\n

Body

<!DOCTYPE html>

<html>

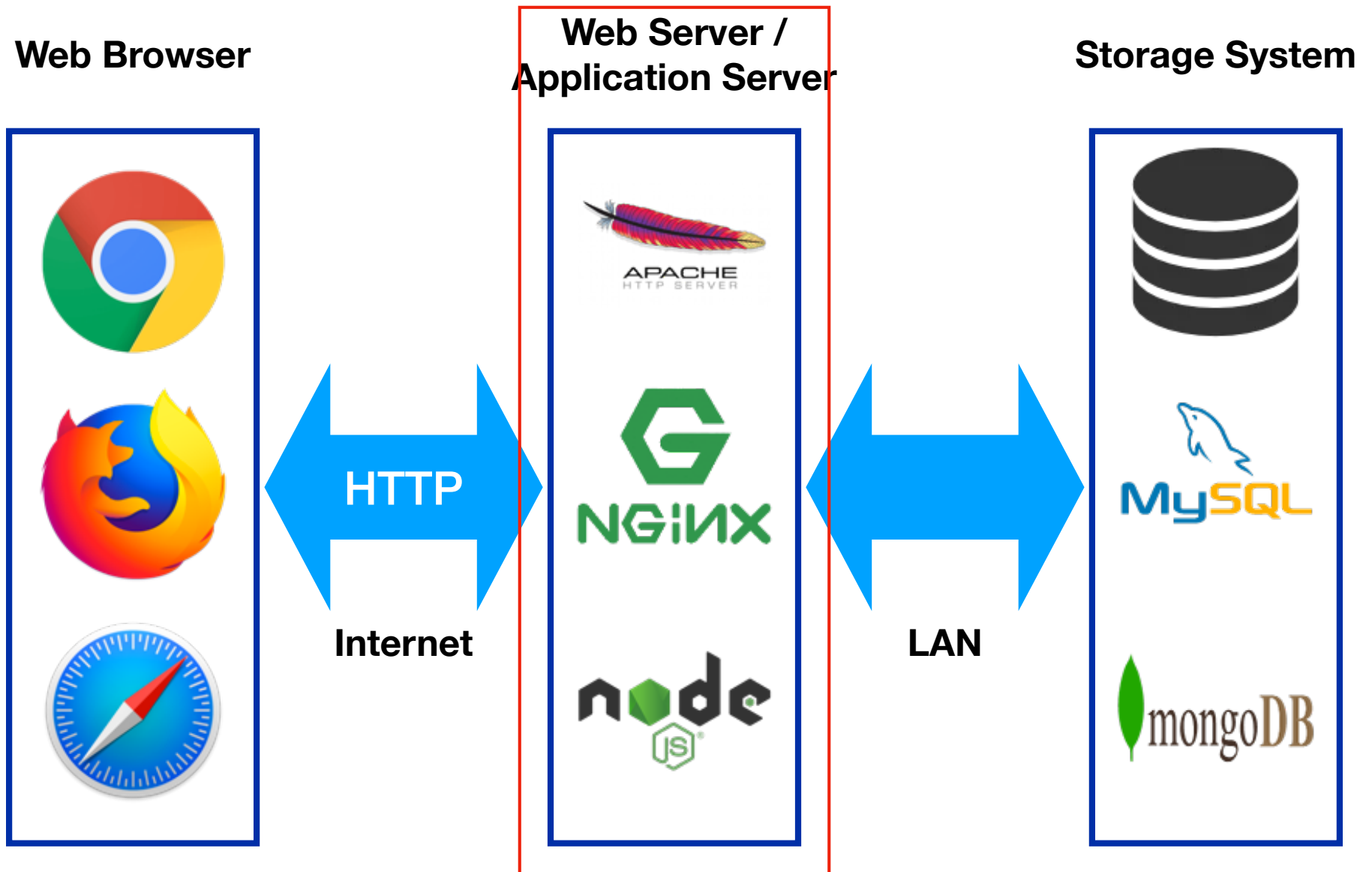
...

</html>

Common HTTP Response Status Codes

Code	Status	Description
1xx	Informational	The request was received, continuing process
200	OK	Success
301	Moved Permanently	Permanent redirection
307	Temporary Redirect	Try URI in the Location field this time
400	Bad Request	Malformed request
401	Unauthorized	Require user authentication
403	Forbidden	Refuse to fulfill the request
404	Not Found	
500	Internal Server Error	Something unexpected (Error) happened
501	Not Implemented	Does not support the requested functionality
503	Service Unavailable	Unable to handle the request due to overloading or maintenance

Web Application Architecture



Web Servers

- Both Web Browsers and Web Servers speak HTTP
- Web Servers: Receive HTTP requests and send HTTP responses
-  A web server loops forever doing
 - Accept TCP connection from a client (browser)
 - Read HTTP request from the connection
 - Process the request
 - Write HTTP response to the connection
 - Close the TCP connection

Serving Static Documents

- Process “GET /index.html HTTP/1.1”

```
int fd = open("index.html");
```

```
int len = read(fd, buffer, sizeofFile(fd));
```

通过TCP连接将HTTP响应头(Header)发送给客户端

```
( write(tcpConnection, httpResponseHeader, headerSize); )
```

```
write(tcpConnection, buffer, len);
```

将缓冲区中的文件内容通过TCP连接发送给客户端

```
close(fd);
```

tcpConnection 已建立的TCP连接套接字（客户端与服务器的通信通道）。

httpResponseHeader 组装好的HTTP响应头字符串（包含状态码、内容类型等元数据）。

headerSize 响应头的字节长度（确保完整发送，不截断）。

就像寄快递时，先写一张运单（响应头）说明包裹内容、重量、收件人，再寄出包裹本身（文件内容）

实际发送的 **httpResponseHeader** 内容类似这样：

http

复制

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1024
Connection: keep-alive
```

• 每一行的作用：

- **HTTP/1.1 200 OK**：协议版本 + 状态码（成功）。
- **Content-Type: text/html**：告诉浏览器返回的是HTML文件。
- **Content-Length: 1024**：声明后续数据长度（方便浏览器预分配内存）。
- **Connection: keep-alive**：保持TCP连接复用（HTTP/1.1默认行为）。

Generating Dynamic Content

- Common Gateway Interface (CGI)

CGI定义了Web服务器（如Apache/Nginx）与外部程序（如Python/PHP脚本）交互的规则

- A **protocol** for **web servers** to **execute programs like calling Command-line Interface (CLI) programs (in another process)** to generate web pages **dynamically**

- Process “GET /index.cgi HTTP/1.1”

```
runProgram(index.cgi, tcpConnection, env, ...);
```

index.cgi : 待执行的CGI程序（如Perl/Python脚本）。

topConnection : TCP连接套接字，用于返回响应。

env : 环境变量（包含 **QUERY_STRING=name=Alice** 等请求信息）。

特性	静态内容 (如PPT前一页)	动态内容 (本页)
生成方式	直接返回磁盘文件	由程序实时生成 (如数据库查询、用户输入处理)
典型文件	<code>.html</code> , <code>.css</code> , <code>.jpg</code>	<code>.cgi</code> , <code>.php</code> , <code>.py</code>
性能	响应快 (无需计算)	延迟高 (需执行程序)
示例场景	公司官网展示页	用户登录后的个性化主页

1. Web浏览器发起HTTP请求

- 用户在浏览器中输入一个URL，浏览器会向Web服务器发送一个HTTP请求。
- 例如，用户访问一个动态网页，请求可能会被发送到运行CGI程序的服务器。

2. Web服务器接收请求

- Web服务器（如Apache）接收到HTTP请求后，会根据请求的URL和配置，决定是否需要调用CGI程序来处理这个请求。
- 如果需要调用CGI程序，服务器会将请求的相关信息（如请求方法、请求参数、环境变量等）传递给CGI程序。

3. CGI程序处理请求

- CGI程序通过标准输入（STDIN）接收请求数据，并通过环境变量（ENV_VAR）获取其他相关信息（如请求方法、客户端IP等）。
- CGI程序根据接收到的数据进行处理，生成响应内容。
- 在示例中，展示了一个简单的Perl CGI程序，该程序的作用是打印出其环境变量。

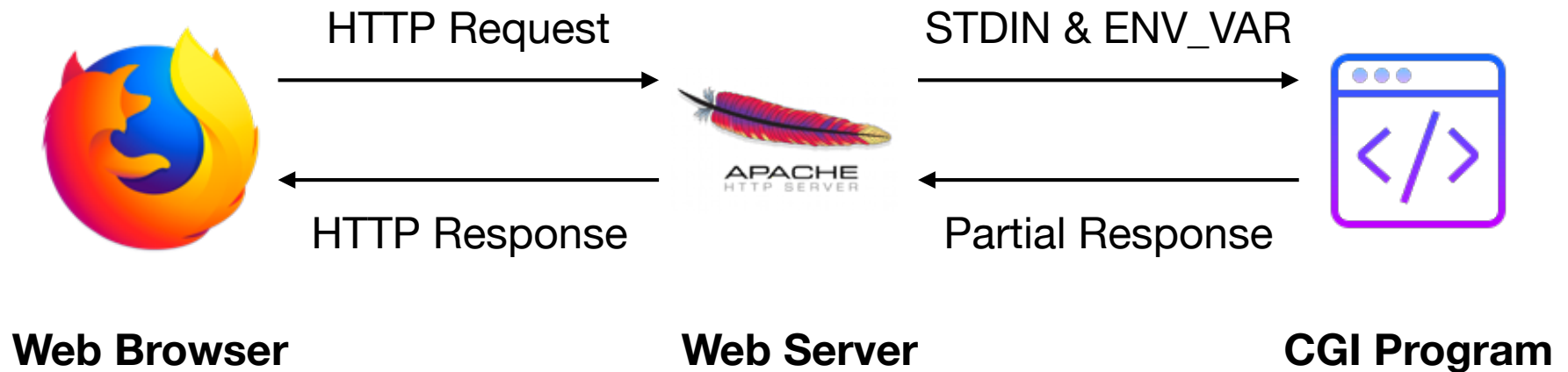
4. CGI程序返回响应

- CGI程序将生成的响应内容通过标准输出（STDOUT）发送回Web服务器。
- 响应内容通常包括HTTP头部信息（如 `Content-type` ）和实际的响应数据。

5. Web服务器发送HTTP响应

- Web服务器接收到CGI程序的响应后，会将其封装成一个完整的HTTP响应，发送回客户端浏览器。
- 浏览器接收到响应后，会根据响应内容进行渲染或进一步处理。

How CGI Programs Work?



这是Shebang行，指定了脚本的解释器路径，这里是Perl解释器

```
#!/usr/bin/perl
```

这是一个Perl文档注释块的开始，用于描述程序的功能或文档信息

```
=head1 DESCRIPTION
```

这是对程序功能的简单描述，说明该CGI程序的作用是打印其环境变量

```
env - a CGI program that just prints its environment
```

结束文档注释块

```
=cut
```

```
print "Content-type: text/plain\n\n";
```

输出HTTP响应头部信息，指定内容类型为纯文本

遍历环境变量%ENV中的所有键（变量名），并按字母顺序排序

```
for my $var ( sort keys %ENV ) {  
    printf "%s = \"%s\"\n", $var, $ENV{$var};  
}
```

对于每个环境变量，使用printf函数格式化输出变量名和对应的值。

%s是字符串占位符，%ENV{\$var}获取环境变量的值。

HTML Forms (表单)

- A **form** is a way to **collect data** from the client 从用户端收集数据的容器
- A form has **form controls**, such as **text fields**, **buttons**, **checkboxes**, **options**, **range controls**, or **color pickers**
 - It is a way to create complex HTTP request
- The **action** attribute contains the URI to submit the HTTP request action 属性
 - Default is the current URI
- The **method** attribute is the HTTP method to submit the request method
 - **GET** or POST

表单提交时会自动构造HTTP请求（GET/POST），将用户输入的数据发送到服务器

控件类型	示例HTML代码	用途
文本输入框	<code><input type="text" name="username"></code>	输入单行文本（如用户名）
复选框	<code><input type="checkbox" name="agree"></code>	多选项选择（如同意条款）
单选按钮	<code><input type="radio" name="gender" value="male"></code>	单选项选择（如性别）
提交按钮	<code><input type="submit" value="提交"></code>	触发表单提交动作
文件上传	<code><input type="file" name="avatar"></code>	上传本地文件

1. 直观对比：GET vs POST

GET请求（数据在URL中）

http复制

```
GET /search?q=hello&page=1 HTTP/1.1
Host: example.com
```

- 数据位置：直接暴露在URL的查询字符串（`?q=hello&page=1`）。
- 可见性：用户可通过地址栏看到数据，可能被浏览器历史记录。

POST请求（数据在Body中）

http复制

```
POST /login HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 28

username=admin&password=123456
```

- 数据位置：隐藏在请求正文（`username=admin&password=123456`）。
- 不可见性：地址栏仅显示路径（`/login`），数据需抓包工具才能查看。

POST请求的Body支持多种编码方式，由 `Content-Type` 头指定：

编码格式	特点	示例场景
<code>application/x-www-form-urlencoded</code>	默认格式，数据为 <code>key=value</code> 对（如 <code>name=John&age=30</code> ），特殊字符转义。	文本表单提交（登录、注册）
<code>multipart/form-data</code>	分块传输，每部分含描述头，支持二进制文件（如图片、PDF）。	文件上传
<code>application/json</code>	直接提交JSON格式数据（现代API常用）。	RESTful API交互

HTML Forms (Cont.)

- Child input tags of a form are transformed into either HTTP query string or part of the HTTP request body
 - **GET**: data is passed in the **query**
 - **POST**: data is passed in the **body**
- The data is encoded as either “application/x-www-form-urlencoded” or “multipart/form-data”
 - GET always uses “application/x-www-form-urlencoded”
 - POST depends on the **enctype attribute** of the form. The default is “application/x-www-form-urlencoded”
 - “multipart/form-data” is mainly used to upload files

HTML Forms (Cont.)

名称 - 值对

- Data is sent as name-value pairs
 - The data is from the `<input>` and other child tags
- Name is taken from the input tag's `name` attribute
 - name属性决定数据的“键”（key），是 若控件无name属性，
服务端获取数据的依据 其数据不会被提交
- Value is taken either from the input tag's `value` attribute, or the user-supplied input
 - Empty string if neither is present

1. 核心概念：名称-值对 (Name-Value Pairs)

- **作用：**表单提交时，所有用户输入的数据会被组织成 **name=value** 的形式，这是HTTP协议传输表单数据的标准格式。

- **示例：**

若表单中有以下输入框：

html

复制

```
<input type="text" name="username" value="默认值">  
<input type="password" name="pwd">
```

▶ 运行 HTML

用户输入后提交时，数据会被转换为：

http

复制

username=默认值&pwd=用户输入的密码

User Input - GET Request

```
<form method="GET" action="form.cgi">
```

Username:

```
<input type="text" name="user"></input><br>
```

Language:

```
<input type="text" name="lang"></input><br>
```

```
<input type="submit"></input>
```

```
</form>
```

Username: alex

Language: enUS

Submit

http://www.example.com/form.cgi?user=alex&lang=enUS

Query String

Method	URL	Version
--------	-----	---------

GET	/form.cgi?user=alex&lang=enUS	HTTP/1.1
-----	-------------------------------	----------

Headers

Host: www.example.com

User-Agent: Mozilla/5.0

Accept: text/html, */*

Accept-Language: en-us

Accept-Charset: ISO-8859-1,utf-8

Connection: keep-alive

\r\n

Body

User Input - POST Request

```
<form method="POST" action="form.cgi">
```

Username:

```
<input type="text" name="user"></input><br>
```

Language:

```
<input type="text" name="lang"></input><br>
```

```
<input type="submit"></input>
```

```
</form>
```

Username: alex

Language: enUS

Submit

http://www.example.com/form.cgi

Method	URL	Version
--------	-----	---------

POST	/form.cgi	HTTP/1.1
------	-----------	----------

Headers

Host: www.example.com

User-Agent: Mozilla/5.0

Accept: text/html, */*

Accept-Language: en-us

Content-Type: application/x-www-form-urlencoded

Content-length: 19

\r\n

Body

user=alex&lang=enUS

(request body)

PHP: Hypertext Processor - PHP

- A scripting language that can be embedded in HTML pages to generate dynamic content
- PHP code is executed on the **server side** when the page containing the code is requested
- A common setup is a LAMP system, which is the composition of
 - Linux
 - Apache
 - MySQL
 - PHP

Credits

- Some slides are based on the slides of Giovanni Vigna